

Polymorphism — One Interface, Many Behaviors

Review worksheet — `virtual`, dynamic binding, `override`, slicing, virtual destructors, abstract classes — builds on Worksheets 04 & 05

Name _____

Date _____

Section _____

part 1

The Problem We Left Off With

picks up W05 Part 7

Worksheet 05 ended on a cliffhanger: a `Vehicle*` pointing at a `Car` still ran `Vehicle::describe()`. Here is that code again, with a pointer *and* a reference:

```
class Vehicle {
public:
    void describe() { cout << "A vehicle.\n"; }
};
class Car : public Vehicle {
public:
    void describe() { cout << "A car.\n"; }
};

int main() {
    Car c;
    Vehicle* vp = &c;    // base-class POINTER to a Car
    Vehicle& vr = c;     // base-class REFERENCE to a Car

    c.describe();        // (a)
    vp->describe();       // (b)
    vr.describe();       // (c)
}
```

QUESTION 1

PREDICT THE OUTPUT — TWICE

CALL	AS WRITTEN ABOVE	AFTER CHANGING VEHICLE TO VIRTUAL VOID DESCRIBE()
(a) <code>c.describe();</code>	_____	_____
(b) <code>vp->describe();</code>	_____	_____
(c) <code>vr.describe();</code>	_____	_____

Adding one keyword changed exactly two of the three lines. Which call did *not* change, and why not?

QUESTION 2

FILL IN THE BLANKS

1. Without `virtual`, the compiler picks which `describe()` to run from the pointer's _____ type, and it decides at _____ time. This is called _____ binding.

2. With `virtual`, the choice comes from the type of the _____ object, and it is decided at _____ time. This is called _____ binding.

3. `virtual` only makes a difference when the call goes through a _____ or a _____.

4. **Polymorphism** (Greek for “many _____”) means one call like `vp->describe()` can produce different behavior depending on the object.

Word bank

actual compile declared dynamic forms pointer reference run static

Polymorphism — One Interface, Many Behaviors

Review worksheet — `virtual`, dynamic binding, `override`, slicing, virtual destructors, abstract classes — builds on Worksheets 04 & 05

part 2

Two Kinds of Polymorphism

builds on W02/W04 overloading

You have already used polymorphism without calling it that. Overloading `print(int)` and `print(string)` gives one name several behaviors. The difference is **when** the right version gets chosen.

KIND	CHOSEN WHEN?	TOOLS	NEEDS INHERITANCE?
Static (compile-time)	While compiling, from the argument types the compiler can see	function overloading, operator overloading, templates	No
Dynamic (run-time)	While running, from the actual object behind a pointer or reference	<code>virtual</code> functions + base-class pointers/references	Yes

QUESTION 3 — LABEL EACH CALL

COMPILE TIME / RUN TIME

For each call, is the version that runs chosen at **compile time** or at **run time**?

`print(5);` where `print(int)` and `print(string)` both exist

`Fraction c = a + b;` using the `operator+` you wrote in W04

`maxOf(3, 7)` and `maxOf(2.5, 1.5)` with `template<typename T> T maxOf(T, T)`

`for (auto& s : shapes) s→draw();` where `draw()` is `virtual`

`Vehicle* v = new Car(); v→describe();` where `describe()` is **not** virtual

`c→attack();` where `c` came from `makeCharacter()` and `attack()` is `virtual`

QUESTION 4

SHORT ANSWER

```
unique_ptr<Character> makeCharacter(string name) {
    switch (rand() % 3) {
        case 0: return make_unique<Warrior>(name);
        case 1: return make_unique<Wizard>(name);
        default: return make_unique<Rogue>(name);
    }
}
```

Why can't the *compiler* know which `attack()` to call on the result of `makeCharacter()` ? What does that force the language to do instead?

Polymorphism — One Interface, Many Behaviors

Review worksheet — `virtual`, dynamic binding, `override`, slicing, virtual destructors, abstract classes — builds on Worksheets 04 & 05

part 3

virtual, override, and the Signature Rules

builds on W05 overriding

A derived function only **overrides** a base function when the base version is `virtual` and the derived version matches it *exactly*: same name, same parameter types, same `const`. Get any of those wrong and you've written a brand-new function that just *hides* the base one, and the compiler won't complain.

Writing `override` after the derived function asks the compiler to check for you: “I mean to replace a virtual base method. Error if I didn't.” Also, once a function is `virtual` in the base class, it stays virtual in every derived class, whether or not the derived class repeats the word.

```
class Character {
public:
    virtual ~Character() {}
    virtual void attack();
    virtual void printName() const;
    virtual void heal(int amount);
    void rest();                // NOT virtual
};

class Wizard : public Character {
public:
    void attack() override;      // (1)
    void printName() override;   // (2)
    void heal(double amount) override; // (3)
    void rest() override;        // (4)
    void Attack() override;      // (5)
    void heal(int amount);        // (6) no 'override' keyword
};
```

QUESTION 5

JUDGE EACH LINE ON ITS OWN

Treat each numbered line as the *only* function in `Wizard`. Write **OK** if it compiles and overrides, or **ERROR** plus the reason.

LINE	OK / ERROR	REASON (IF ERROR)
(1)		
(2)		
(3)		
(4)		
(5)		
(6)		

QUESTION 6

THE SILENT BUG

Suppose you delete the word `override` from line (2), so it now compiles. What does this print, and why?

Character* p = new Wizard();
p->printName(); // Character::printName() or Wizard::printName()?

Polymorphism — One Interface, Many Behaviors

Review worksheet — `virtual`, dynamic binding, `override`, slicing, virtual destructors, abstract classes — builds on Worksheets 04 & 05

part 4 Pointers, References, and Slicing

Dynamic binding only happens through a **pointer** or a **reference**. If you *copy* a `Wizard` into a plain `Character` variable, only the `Character` part gets copied and the `Wizard` part is thrown away. This is called **object slicing**. Once an object is sliced, `virtual` can't bring the lost part back.

```
class Character {
protected:
    string name;
public:
    Character(string n) : name(n) {}
    virtual ~Character() {}
    virtual void printName() { cout << name << endl; }
};
class Wizard : public Character {
public:
    Wizard(string n) : Character(n) {}
    void printName() override { cout << "The magnificent " << name << endl; }
};
class Warrior : public Character {
public:
    Warrior(string n) : Character(n) {}
    void printName() override { cout << "The powerful " << name << endl; }
};

void byRef(Character& c) { c.printName(); }
void byPtr(Character* c) { c->printName(); }
void byVal(Character c) { c.printName(); }

int main() {
    Wizard merlin("Merlin");
    Warrior conan("Conan");

    byRef(merlin);                // (a)
    byPtr(&conan);                 // (b)
    byVal(merlin);                 // (c)

    vector<Character> copies = {merlin, conan};
    copies[1].printName();         // (d)

    vector<Character*> party = {&merlin, &conan};
    party[1]->printName();         // (e)
}
```

QUESTION 7

PREDICT THE OUTPUT

(a) _____

(b) _____

(c) _____

(d) _____

(e) _____

QUESTION 8

SHORT ANSWER

Lines (c) and (d) both print the plain name even though `printName()` is virtual. What do the two lines have in common? What is the one-character change to `byVal` that fixes (c)?

Polymorphism — One Interface, Many Behaviors

Review worksheet — `virtual`, dynamic binding, `override`, slicing, virtual destructors, abstract classes — builds on Worksheets 04 & 05

part 5

One Loop, Many Types

the whole point

This is why polymorphism is worth the trouble: different kinds of objects go into **one container**, and **one loop** handles all of them without ever asking “what kind are you?” A derived class that doesn’t override a virtual function just uses the version it inherited.

```
class Character {
protected:
    string name;
public:
    Character(string n) : name(n) {}
    virtual ~Character() {}
    virtual void printName() { cout << name << endl; }
    virtual void attack()    { cout << name << " swings wildly." << endl; }
};
class Warrior : public Character {           // overrides BOTH
public:
    Warrior(string n) : Character(n) {}
    void printName() override { cout << "The powerful " << name << endl; }
    void attack()    override { cout << name << " cleaves with an axe!" << endl; }
};
class Wizard : public Character {           // overrides printName only
public:
    Wizard(string n) : Character(n) {}
    void printName() override { cout << "The magnificent " << name << endl; }
};
class Rogue : public Character {           // overrides attack only
public:
    Rogue(string n) : Character(n) {}
    void attack() override { cout << name << " stabs from the shadows!" << endl; }
};

int main() {
    vector<unique_ptr<Character>> party;
    party.push_back(make_unique<Warrior>("Conan"));
    party.push_back(make_unique<Wizard>("Merlin"));
    party.push_back(make_unique<Rogue>("Vex"));
    party.push_back(make_unique<Character>("Bob"));

    for (auto& c : party) {
        c->printName();
        c->attack();
    }
}
```

QUESTION 9			TRACE THE LOOP
OBJECT	C->PRINTNAME() PRINTS	C->ATTACK() PRINTS	
Warrior("Conan")			
Wizard("Merlin")			
Rogue("Vex")			
Character("Bob")			

Polymorphism — One Interface, Many Behaviors

Review worksheet — `virtual`, dynamic binding, `override`, slicing, virtual destructors, abstract classes — builds on Worksheets 04 & 05

part 5 One Loop, Many Types (continued)

Here is the same battle written *without* polymorphism. Every character is one struct with a `type` string, and every behavior has to check that string:

```
struct Character {
    string type;    // "warrior", "wizard", "rogue"
    string name;
};

void attack(const Character& c) {
    if (c.type == "warrior")    cout << c.name << " cleaves with an axe!\n";
    else if (c.type == "wizard")    cout << c.name << " swings wildly.\n";
    else if (c.type == "rogue")    cout << c.name << " stabs from the shadows!\n";
}

void printName(const Character& c) {
    if (c.type == "warrior")    cout << "The powerful " << c.name << "\n";
    else if (c.type == "wizard")    cout << "The magnificent " << c.name << "\n";
    else                          cout << c.name << "\n";
}

// ...and defend(), levelUp(), save(), draw(), ... each with its own if/else chain
```

QUESTION 10ADDING A BARD

The game needs a new `Bard` whose attack is “`plays a very loud lute!`”. List what you would have to write or edit in each design.

POLYMORPHIC VERSION (PART 5)	IF/ELSE VERSION (ABOVE)

In the polymorphic version, does the `for` loop in `main()` change at all? _____ What goes wrong in the if/else version if you forget one function?

QUESTION 11 — WRITE THE CLASSBARD

Write `Bard` so it works in the Part 5 party. Derive publicly, chain the constructor, and override `attack()` with `override`. Keep the inherited `printName()`.

```
class Bard :
```

```
};
```

Write the one line that adds a Bard named `"Jaskier"` to `party` :

Polymorphism — One Interface, Many Behaviors

Review worksheet — `virtual`, dynamic binding, `override`, slicing, virtual destructors, abstract classes — builds on Worksheets 04 & 05

part 6 Virtual Destructors

builds on W05 destruction order

A destructor is looked up like any other member function. When you `delete` a derived object through a base pointer and the base destructor is **not** virtual, the compiler goes by the pointer's type and calls only the base destructor. The derived destructor never runs. (Officially the behavior is *undefined*. In practice, whatever the derived part owned leaks.)

```
class Base {
public:
    ~Base() { cout << "~Base "; }           // not virtual
};

class Derived : public Base {
    int* data;
public:
    Derived() : data(new int[100]) {}
    ~Derived() { cout << "~Derived "; delete[] data; }
};

int main() {
    Base* p = new Derived();
    delete p;
}
```

QUESTION 12

PREDICT & FIX

1. What does the program print? _____
2. What memory is leaked? _____
3. The one-word fix, and exactly where it goes: _____
4. After the fix, what does it print? _____ (Recall the W05 destruction order.)

QUESTION 13

SHORT ANSWER

With the *original* non-virtual `~Base()`, is this version a problem? Why or why not?

```
int main() {
    Derived d;
} // d goes out of scope here
```

QUESTION 14

THE RULE

Finish the rule of thumb: “If a class has _____, give it a _____.”

Polymorphism — One Interface, Many Behaviors

Review worksheet — `virtual`, dynamic binding, `override`, slicing, virtual destructors, abstract classes — builds on Worksheets 04 & 05

part 7 Abstract Classes & Pure Virtual Functions

What does a plain `Shape` look like? What's its area? There's no good answer, because `Shape` is an idea and not a real thing. A **pure virtual function** (`= 0`, no body) says: “every derived class *must* provide this.” A class with at least one pure virtual function is **abstract**, so you can't create an object of it. You *can* still use pointers and references to it, and it can still have data, constructors, and ordinary methods. A derived class that overrides every pure virtual function is **concrete**. One that misses any is abstract too.

```
class Shape {
protected:
    string label;
public:
    Shape(string l) : label(l) {}
    virtual ~Shape() {}
    virtual double area() const = 0;           // pure virtual
    string name() const { return label; }     // ordinary method
};

class Circle : public Shape {
    double r;
public:
    Circle(double r) : Shape("circle"), r(r) {}
    double area() const override { return 3.14159 * r * r; }
};

class Blob : public Shape {
public:
    Blob() : Shape("blob") {}
    // no area()
};
```

QUESTION 15

COMPILES? Y / N

```
Shape s("thing");
```

```
Shape* p = new Circle(2.0);
```

```
Circle c(1.0); Shape& ref = c;
```

```
Blob b;
```

```
Circle c(1.0); cout << c.name();
```

```
vector<Shape*> shapes;
```

For the `Blob` line: what is the error, and what would make `Blob` concrete?

QUESTION 16

FILL IN THE BLANKS

1. A function declared `virtual double area() const = 0;` is called a _____ function.

2. An abstract class whose members are *only* pure virtual functions (no data) is often called an _____.

Polymorphism — One Interface, Many Behaviors

Review worksheet — `virtual`, dynamic binding, `override`, slicing, virtual destructors, abstract classes — builds on Worksheets 04 & 05

part 7 Abstract Classes (continued)

QUESTION 17 — WRITE THE CLASS

RECTANGLE

Using `Shape` from the previous page, write a concrete `Rectangle` class. Give it private `width` and `height`. Its constructor takes both and chains to `Shape` with the label `"rectangle"`. It overrides `area()`. Watch the `const`.

```
class Rectangle :
```

```
};
```

QUESTION 18 — WRITE THE FUNCTION

TOTALAREA

Write a function that returns the sum of the areas of every shape in the vector. It must work for `Circle`, `Rectangle`, and any shape class someone writes *next year*, without being edited.

```
double totalArea(const vector<Shape*>& shapes) {
```

```
}
```

Given `Circle c(1.0);` and `Rectangle r(2.0, 3.0);`, what does `totalArea({&c, &r})` return? _____

Polymorphism — One Interface, Many Behaviors

Review worksheet — `virtual`, dynamic binding, `override`, slicing, virtual destructors, abstract classes — builds on Worksheets 04 & 05

part 8 Find the Error

four traps

CASE A COMPILES, BUT...

```
class Animal {
public:
    void speak() { cout << "...\\n"; }
};
class Dog : public Animal {
public:
    virtual void speak() { cout << "Woof!\\n"; }
};

Dog d;
Animal& a = d;
a.speak();           // line X
```

What does line X print? _____ Why doesn't `virtual` help here?

CASE B COMPILES? Y / N

```
class Shape {
public:
    virtual ~Shape() {}
    virtual double area() const = 0;
};
class Square : public Shape {
    double side;
public:
    Square(double s) : side(s) {}
    double area() { return side * side; }
};

Square sq(3.0); // line Y
```

Does line Y compile? _____ What's wrong, and which keyword would have caught it one line earlier?

CASE C COMPILES & RUNS, BUT...

```
int main() {
    vector<Character*> party;
    party.push_back(new Wizard("Merlin"));
    party.push_back(new Warrior("Conan"));
    for (auto c : party)
        c->attack();
}
```

What's the problem? Give two different fixes.

CASE D COMPILES? Y / N

```
class Wizard : public Character {
public:
    Wizard(string n) : Character(n) {}
    void attack() override { /* ... */ }
    void castSpell() { /* ... */ }
};

Character* c = new Wizard("Merlin");
c->attack();
c->castSpell(); // line Z
```

Does line Z compile? _____ Why?

TAKEAWAY

CASES A & D IN ONE LINE

The one sentence to remember: the *pointer's* type decides what you're **allowed to call**. The *object's* type decides **which version runs**, but only if the function is `virtual`.

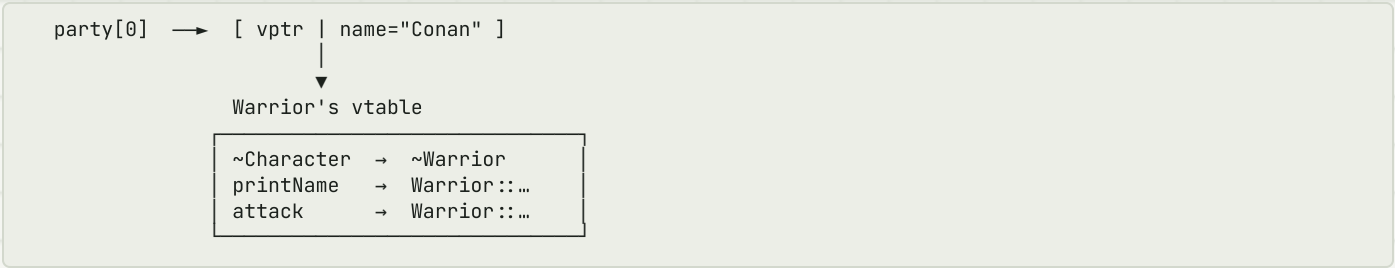
Polymorphism — One Interface, Many Behaviors

Review worksheet — `virtual`, dynamic binding, `override`, slicing, virtual destructors, abstract classes — builds on Worksheets 04 & 05

part 9

Under the Hood: the vtable

How does `c→attack()` find the right function at run time? Every class that has virtual functions gets **one** hidden **vtable**, a table of function pointers with one slot per virtual function. Every *object* of that class carries a hidden **vp**tr that points to its class's vtable. The vptr is set when the object is constructed. A virtual call follows the vptr, looks up the slot, and calls whatever it finds there.



QUESTION 19

FILL IN THE VTABLES FOR PART 5'S CLASSES

Write which class's version each slot points to (e.g. `Character::` or `Rogue::`).

VTABLE FOR	PRINTNAME SLOT	ATTACK SLOT
<code>Character</code>		
<code>Warrior</code>		
<code>Wizard</code>		
<code>Rogue</code>		

QUESTION 20

TRUE / FALSE

Every object gets its own private copy of its class's vtable. _____

Adding the first virtual function to a class usually makes each object a little bigger. _____

Non-virtual functions like `Shape::name()` are called through the vtable. _____

Which vtable an object uses depends on the pointer's declared type. _____

A virtual call does a little extra work (following pointers) that a normal call skips. _____

cheat sheet

Polymorphism at a Glance

CONCEPT	TYPICAL SYNTAX	PURPOSE
Virtual function	<code>virtual void attack();</code>	Pick the version from the <i>actual</i> object, at run time
Override check	<code>void attack() override;</code>	Compile error if it doesn't really override
Base pointer / reference	<code>Character* c</code> · <code>Character& c</code>	The only way dynamic binding happens
Slicing	<code>Character c = wizard;</code>	A copy keeps only the base part (avoid it)
Virtual destructor	<code>virtual ~Character() {}</code>	<code>delete</code> through a base pointer runs the derived destructor too
Pure virtual	<code>virtual double area() const = 0;</code>	Makes the class abstract; derived classes must implement it
Owning container	<code>vector<unique_ptr<Character>></code>	Mixed types in one container, deleted for you automatically